Quantification of frailty via the lens of feasible output

shows that ankle plantar flexors are most crucial to stair ascent

Eric Kwei [1], John Kim[1], Sanskriti Gurappa[1], Yamato Irie[1], Zijie Zhu[1], *Francisco Valero-Cuevas[1,2]

[1]Biomedical Engineering, [2]Biokinesiology & PT, University of Southern California

*valero@usc.edu

1. Abstract

Clinical measures of frailty focus on decline in physiology and mobility but often lack neuromechanical specificity. We use a computational approach to quantify the relationship between muscle strength and the mechanical demands of stair ascent, as it is an early indicator of functional decline in older adults, and may serve as a sensitive early detector of frailty. We use Feasibility Theory[1] to evaluate the impact of muscle weakening on the ability to meet the vertical ground-reaction forces (GRFs) required throughout the cycle of stair ascent. We applied published kinematic data[2] during stair ascent to a normative adult OpenSim lower-limb musculoskeletal model to predict the feasible force set (FFS) polytope at the leading foot during the ascent cycle as a function of optimal muscle fiber length, moment arms, pennation angles, tendon slack lengths, and maximum isometric force. Considering stepping as quasi-static, we computed moment-arms, Jacobians, and the maximum muscle force to calculate the FFS of the leg[3] during the step climbing cycle. This allowed us to quantify "frailty" as the distance of the GRF needed throughout in the cycle to the edge of the FFS: if the FFS shrinks below the necessary GRF at any point in the cycle, then stair climbing becomes infeasible. Results: The shape and size of the FFS polytope changes across the ascent cycle, and Monte Carlo simulations weakening individual muscles show some muscles are particularly critical. Even modest (10–20%) strength reductions in the plantar flexor from normative values disproportionately shrink the feasible set's vertical component below the required GRF during certain phases of the ascent, resulting in failure, and suggesting limited robustness to plantar flexor muscle weakening. Defining frailty as a loss of feasible mechanical output allows comparison to task demands. This can help identify specific muscles and gait phases most vulnerable to sarcopenia in healthy aging or disease. This framework offers quantifiable thresholds for frailty to inform targeted strengthening interventions for older adults.

2. Introduction

Frailty, a condition commonly seen in older adults and people with chronic illness, is characterized by reduced strength and mobility, and impairs daily tasks such as walking, standing up, or climbing stairs, increasing the risk of falls, injury, hospitalization, and loss of independence[6]. While currently diagnosed via observations or questionnaires, these methods may not reflect the underlying physical limitations causing functional decline[7]. Stair ascent is a high-demand activity, requiring coordinated muscle action and sufficient force production from the lower limb[8]. Many individuals who can walk on level ground report difficulty with stair climbing early in the progression of frailty[9], suggesting that it may be a sensitive task for revealing reduced physical reserve. Yet, the specific mechanical failure points remain unclear.

Biomechanical modeling links muscle properties to task demands to assess feasibility. In this study, we use a musculoskeletal framework to evaluate lower-limb force capacity during stair ascent. By computing posture-dependent feasible force sets and comparing them to required ground reaction forces (GRF), we identify whether a task is mechanically feasible. Using Monte Carlo simulations to account for population variability, we aim to identify critical muscles and phases of stair ascent most sensitive to strength loss, establishing whether failure to meet task demands can serve as a biomechanical indicator of frailty.

3. Model considerations and assumptions

Joint torque is modeled as moment arms multiplied by maximal muscle forces (***Eq 1)***. We use existing datasets for joint angles[2,] moment arms[4] and optimal muscle fiber lengths[5]. GRF is simulated using an *OpenSim* lower limb model (*gait2392)*, with joint moments computed via inverse dynamics. Verification is performed by comparing simulated joint moments recorded from the actual dataset in the *Walk to Run* task.

$$\boldsymbol{\tau} = \boldsymbol{R} \times \boldsymbol{F_0} \qquad \textit{Eq 1}$$

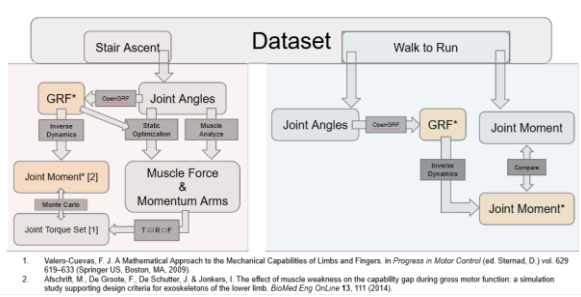

Figure 1 Simulation model flow. GRF and joint moments were simulated on OpenSim using joint angles from data set[2]. Verification was done using walk-to-run dataset because stair ascent did not include measured GRF and joint moment.

Preprocessing

Subject weight and marker positions from Reznick et al. are scaled in OpenSim using distance ratios of custom markers (hip, knee, ankle, toes) to ensure simulated GRF and joint moments match experimental parameters.

4. Development of equations

Maximal forces are calculated using the following equations:

$$F_0 = \sigma_{max} \times PCSA \qquad \textbf{\textit{Eq 2}}$$

σ_{max}: maximal stress (35 N/cm^2)

$$PCSA = \frac{m \cos \alpha^{opt}}{\rho \, l_f^{opt}} \qquad \textbf{\textit{Eq 3}}$$

ρ = muscle density (1.06 g/cm^3); l_f^{opt}= optimal fiber length; α^{opt}= fiber pennation angle; m = mass

5. Design iterations and test trials

Monte Carlo

We use Monte Carlos simulations to evaluate how variability in muscle strength and anatomical parameters influences mechanical feasibility. Task failure is defined as the inability of the feasible force set to meet posture-specific GRF requirements at any point in the stair cycle.

GUI Implementation

To complement the Monte Carlo simulations, we developed a Python-based, interactive GUI[10] to deterministically visualize mechanical feasibility. It integrates moment arms and force capacities to compute the Feasible Torque Set (FTS), projecting it into the Feasible Force Set (FFS) via the Jacobian. By calculating the Minkowski sum of selected muscle generators, the GUI allows us to determine theoretical force limits and tests limb robustness by selectively deactivating muscles (simulating severe sarcopenia) to identify critical contributors to the FTS and FFS polytopes.

6. Results

Stair-ascent GRF simulation (target dataset).

Using stair-ascent kinematics as input, we simulated the time-varying GRF profiles for the left and right feet. The resulting waveforms show an alternating support structure with near-zero force during non-contact periods and substantial loading during stance, consistent with expected gait mechanics during stair negotiation (Fig. 3). These simulated GRFs were used as external loads for the subsequent inverse-dynamics computations in the target stair-ascent task.

Validation of simulated joint moments on level walking (validation dataset).

Since ground-truth joint moments are unavailable for stair ascent, we evaluated the joint-moment estimation accuracy on a level-walking validation dataset that includes reference (ground-truth) joint moments. We computed bilateral hip, knee, and ankle net joint moments using the same pipeline and compared them against the reference signals (Figure 4). Quantitatively, Table 1 summarizes MAE, RMSE, NRMSE, and Pearson correlation (r) for each joint and side. All metrics were computed over the full gait cycle (0–100%), defined from initial contact to the subsequent initial contact, without separating stance and swing phases. Joint moments were not body weight normalized (units: N·m). Overall, agreement was strongest at the hip (Right Hip r=0.863; Left Hip r=0.863), moderate at the knee (Right Knee r=0.472; Left Knee r=0.603), and lowest at the ankle (Right Ankle r=0.239; Left Ankle r=0.474), with the ankle also exhibiting the largest errors (e.g., RMSE up to 104.070 N·m). This pattern is consistent with the greater sensitivity of distal inverse-dynamics moments—particularly at the ankle—to uncertainties in external loading and center-of-pressure estimates.

Application to stair ascent (target dataset).

After validation on level walking, we applied the same GRF-simulation and inverse-dynamics workflow to the stair-ascent dataset to generate the hip, knee, and ankle net joint moments used as the task-required joint demands (NeededTorque) in subsequent feasible-torque and Monte Carlo frailty analyses. Because the stair-ascent dataset lacks ground-truth joint moments, these stair-ascent joint-moment trajectories should be interpreted as model-based estimates derived from simulated external loads and measured kinematics.

Muscle Contributions to Task Failure

Weakness in the ankle plantar flexors was the strongest predictor of torque infeasibility. Logistic regression shows that reduction in soleus strength has the largest effect on failure probability, followed by the medial and lateral gastrocnemius. These muscles consistently show larger coefficients than all the other muscle groups, which indicate their dominant role in determining task feasibility. Correlation analyses support this finding, showing strong negative relationships between plantar flexor strength and failure ratio across the population.

Sensitivity analysis using linear regression further demonstrates that variability in soleus strength produces the largest changes in failure ratio when all muscles are simultaneously varied. In contrast, proximal muscles such as the quadriceps exhibit smaller sensitivities, which suggests that ankle plantar flexor capacity is the dominant constraint in stair ascent.

Frailty Thresholds and Tolerance to Strength Loss

We estimate muscle frailty thresholds by identifying the muscle strength level where the subjects transition from feasible to infeasible performance. The plantar flexors exhibit the lowest thresholds, indicating limited tolerance to strength loss. Even moderate reductions in soleus or gastrocnemius force capacity result in mechanical failure. Other muscle groups tolerate larger reductions, which suggests greater redundancy or compensatory capacity outside the ankle joint.

Phase-Dependent Effects of Frailty

Analysis of torque infeasibility across the stair cycle shows that frailty-related failure is not evenly distributed over time. Failure probability peaks during early stance and late push-off—phases requiring rapid load acceptance and vertical propulsion. Conversely, mid-stance shows minimal difference between frail and non-frail subjects, indicating that specific phases of ascent offer limited mechanical redundancy.

Population-Level Failure Distribution

Failures are most common during 10–20% of the stair cycle (push-off), with a smaller subset of subjects experiencing substantially higher failure rates. This distribution reflects natural inter-individual variability in strength and anatomy, while also highlighting the relatively narrow margin between feasible and infeasible performance for this task.

Interpretation of GUI Results

GUI results (Fig. 5) validate the Monte Carlo sensitivity analysis. While horizontal capacity remains robust after muscle loss, vertical capacity is critically fragile. At push-off (frame 12), deactivating the soleus alone reduces the maximum vertical component of the FFS by ~56.3% (from -2700 N to -1180 N). Removing the entire plantar flexor group (medial/lateral gastrocnemius, soleus, tibialis posterior, flexor digitorum, and flexor hallucis) reduces it by ~89.4% (to -285 N). In both states, the required GRF vector falls substantially outside the feasible polytope, indicating an inability to support body weight and generate necessary propulsion at that frame in the stair ascent cycle.

7. Discussion

Summary

This study frames frailty as a loss of mechanical feasibility during stair ascent. Monte Carlo simulations and visualization from our GUI identified the ankle plantar flexors, particularly the soleus and gastrocnemius, as the primary limiters of performance. Failure is most concentrated during early stance and late push-off, phases demanding high vertical force. These results suggest that plantar flexor strength is a critical determinant of stair ascent capability, and that frailty can be interpreted as a loss of feasible mechanical output.

Limitations

This study relies on a quasi-static, normative model, excluding dynamic effects like inertia and rapid power generation. GRF and joint moments are derived from simulations driven by published kinematics rather than direct measurement, and validation was performed using a separate walk-to-run dataset.

Monte Carlo variations are independent, neglecting anatomical correlations, age-related changes in tendon properties, and neural control strategies such as co-contraction. The frailty classification threshold based on failure ratio (>0.2) is a modeling choice rather than a clinically calibrated criterion. Additionally, the GUI-based muscle removal experiments represent extreme focal weakness and do not account for compensatory adaptations that may occur in vivo. Consequently, these findings are best interpreted as identifying relative mechanical sensitivity and vulnerable phases of stair ascent, rather than subject-specific clinical predictions.

8. Appendix

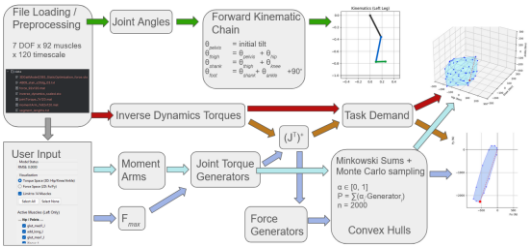

Figure 2 GUI flowchart: Details inputs/outputs and logical process of the Python code to produce FTS/FFS for each frame in ascent cycle.

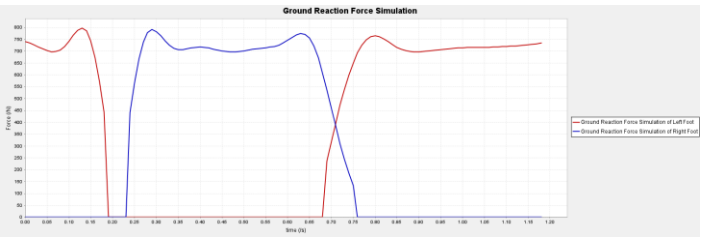

Figure 3. Simulated ground reaction forces (GRFs) during stair ascent: Simulated GRF time series for the left and right feet during the stair-ascent task, showing alternating support with near-zero force during non-contact periods and substantial loading during stance. These GRFs were used as external loads for inverse dynamics joint-moment estimation in the target dataset.

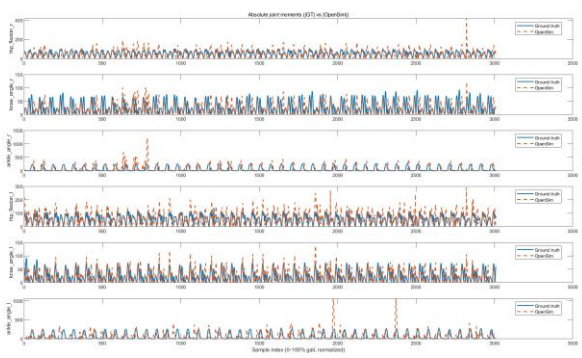

Figure 4. Joint moment verification on the level-walking validation dataset: Comparison of simulated inverse-dynamics joint moments (OpenSim) versus reference (ground-truth) joint moments for bilateral hip, knee, and ankle across the normalized gait cycle (0–100%). This figure provides waveform-level verification of the GRF-simulation + inverse-dynamics pipeline under conditions where ground-truth joint moments are available.

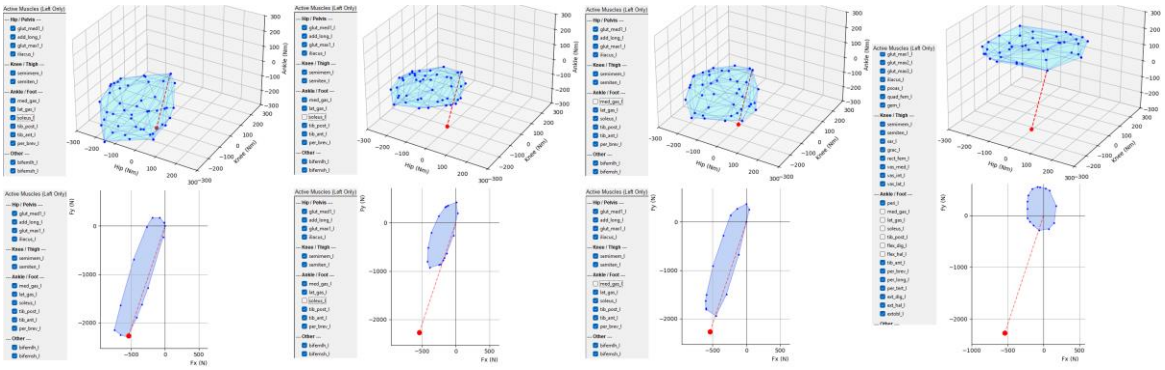

Figure 5 GUI Comparison: Displays FTS/FFS with all muscles included (left), only soleus excluded (left center), only medial gastrocnemius excluded (right center), medial and lateral gastrocnemius, soleus, tibialis posterior, flexor digitorum, and flexor hallucis excluded (right); drastic shrinkage supports Monte Carlo sensitivity analysis—soleus and other plantar flexor muscles are most critical

Table 1. Quantitative accuracy of simulated joint moments on the level-walking validation dataset.

JointDOF	MAE_Nm	RMSE_Nm	NRMSE_percent	PersonR
Right Hip	25.073	35.257	33.075	0.86331
Right Knee	17.199	26.671	28.612	0.47248
Right Ankle	56.147	104.070	38.345	0.23890
Left Hip	26.974	37.603	26.864	0.86281
Left Knee	12.429	19.068	20.548	0.60342
Left Ankle	58.997	98.426	36.264	0.47377

Code 1 Joint Torque Computation

```
%% Joint torque computation (R*F)
% Purpose:
% Compute net joint torque trajectories tau(t) from muscle moment arms and muscle forces
% using the linear superposition model: tau(t) = R(t) * F(t).
%
% Key assumptions (critical for correct interpretation):
% (A1) Moment arms R(t) are obtained from OpenSim Muscle Analysis.
% Units: meters (m). Sign convention follows the OpenSim model definitions for each DoF.
% (A2) Muscle forces F(t) are obtained from the force/optimization pipeline (e.g., Static
Optimization).
% Units: Newtons (N). Forces are assumed to be along each muscle–tendon line of action.
% (A3) Net joint torque is modeled as the sum of individual muscle torques:
% tau_j(t) = sum_i r_{j,i}(t) * F_i(t), i.e., linear additivity across muscles.
% This yields tau(t) in N·m (since m * N = N·m).
% (A4) Muscle ordering MUST be consistent between R and F:
% the i-th column of R(t) must correspond to the same muscle as the i-th entry of F(t).
% Any mismatch in muscle names/order will produce incorrect torques.
% (A5) Time alignment MUST be consistent:
% MomentArm(:,:,t) and Force(:,t) must refer to the same time sample (same nTime and time grid).
% --- Inputs expected to be already loaded/constructed ---
% MomentArm: [nDOF x nMuscles x nTime] (each page is R(t))
% Force: [nMuscles x nTime] (each column is F(t))
% Time: [nTime x 1]
% 1) Sanity check: dimensions must match exactly (same muscle count, same time length)
[nDOF, nMus, nTime] = size(MomentArm);
assert(isequal(size(Force), [nMus, nTime]), ...
'Dimension mismatch: Force must be [nMuscles x nTime] to match MomentArm.');
% 2) Core computation: joint torque at each time sample (tau(:,t)=R(:,:,t)*F(:,t))
JointTorque = zeros(nDOF, nTime);
for t = 1:nTime
R_t = squeeze(MomentArm(:,:,t)); % R(t): [nDOF x nMuscles] (m)
F_t = Force(:,t); % F(t): [nMuscles x 1] (N)
JointTorque(:,t) = R_t * F_t; % tau(t): [nDOF x 1] (N·m)
end
% (Optional) Vectorized form (MATLAB R2020b+):
% JointTorque = squeeze(pagemtimes(MomentArm, reshape(Force, [nMus, 1, nTime]))); % [nDOF x nTime]
```

Code 2 Torque infeasibility and failure ratio

```
%% ============================================================
%% Torque infeasibility and failure ratio
%% ============================================================
% Purpose:
% Determine whether the required net joint torque vector P(t) is achievable by the
% muscle-generated torque set at each time frame, and summarize infeasibility as a
% subject-level failure ratio.
%
% Textual definitions (embedded):
% - Per-muscle torque contribution in the 3-DOF torque space:
% T_i(t) = r_i(t) * F_i(t) (units: N·m)
% where r_i(t) is the moment-arm vector (m) for the selected 3 DOFs and muscle i,
% and F_i(t) is the muscle force capacity at time t (N).
```

```
%
% - Required net joint torque (task demand) from inverse dynamics:
% P(t) (3x1, units: N·m)
%
% - Feasibility test (support-function approximation along the demand direction):
% u(t) = P(t) / ||P(t)|| (unit direction in torque space)
% t_max(t) = Σ_i max(0, T_i(t)^T u(t)) (max achievable magnitude along u)
% A frame is labeled infeasible if:
% ||P(t)|| > t_max(t)
% Interpretation:
% Only muscles that project positively onto the demand direction are assumed
% to be recruited (no reliance on antagonistic cancellation).
%
% - Failure ratio (subject-level):
% fail_ratio = (# infeasible evaluated frames) / (# valid evaluated frames)
% Binary failure label:
% isFail = (fail_ratio > 0.20)
function [fail_ratio, isFail, isFail_eachTime] = evaluate_fail_ratio_simple( ...
MA3_sel, Force_sel, NeededTorque, timeIdxToUse)
fail_threshold = 0.20; % subject fails if infeasible > 20% of evaluated frames
[~, nMusSel, nTime] = size(MA3_sel);
isFail_eachTime = false(1, nTime);
% Ensure time indices are valid and unique
timeIdxToUse = unique(timeIdxToUse(timeIdxToUse >= 1 & timeIdxToUse <= nTime));
for t = timeIdxToUse
% Build per-muscle torque vectors at time t:
% Tv_t(:,i) = r_i(t) * F_i(t)
Tv_t = MA3_sel(:,:,t) .* Force_sel(:,t)'; % [3 x nMusSel], units N·m
P = NeededTorque(:, t); % [3 x 1], units N·m
% Skip invalid/degenerate frames (no direction if ||P|| = 0)
if ~all(isfinite(P)) || norm(P) == 0 || ~any(isfinite(Tv_t(:)))
continue;
end
% Demand direction u = P / ||P||
u = P / norm(P);
% Project each muscle torque onto u: proj_i = Tv_t(:,i)' * u
proj = Tv_t' * u; % [nMusSel x 1], units N·m
% Support-function approximation: sum of positive projections gives t_max along u
t_max = sum(proj(proj > 0)); % scalar, units N·m
% Infeasible if required magnitude exceeds achievable magnitude along same direction
if norm(P) > t_max + 1e-8
isFail_eachTime(t) = true;
end
end
% Compute failure ratio over evaluated frames with finite NeededTorque
validIdx = intersect(timeIdxToUse, find(all(isfinite(NeededTorque),1)));
fail_ratio = sum(isFail_eachTime(validIdx)) / max(numel(validIdx), 1);
isFail = (fail_ratio > fail_threshold);
end
```

Code 3 Muscle frailty threshold (one-muscle weakening)

```
%% ============================================================
%% Muscle frailty threshold (one-muscle weakening)
```

```
%% ============================================================
% Purpose:
% Quantify muscle criticality for a fixed virtual subject by weakening ONE muscle
% at a time and identifying when task infeasibility exceeds a predefined criterion.
%
% Textual definitions (embedded):
% - Single-muscle weakening model for muscle m:
% F_m(t) <- s * F_m(t), where s ∈ [0,1] is the remaining strength (capacity)
% All other muscles remain unchanged.
%
% - Frailty threshold for muscle m:
% threshold(m) = smallest s such that fail_ratio(s) > 0.20
% Interpretation:
% threshold(m) close to 1 => slight weakening triggers failure => muscle is critical
% threshold(m) close to 0 => even severe weakening does not trigger failure => less critical
%
% NOTE:
% scale_steps = 1.0:-0.01:0.0 corresponds to 1% steps (0.01 = 1%), not 10%.
function threshold = compute_muscle_thresholds( ...
MA3_sel_i, Force_sel_i, NeededTorque, timeIdxToUse)
[~, nMusSel, ~] = size(MA3_sel_i);
scale_steps = 1.0:-0.01:0.0; % remaining strength from 100% to 0% in 1% steps
fail_threshold = 0.20; % same subject-level criterion as Appendix B
threshold = zeros(nMusSel,1);
for m = 1:nMusSel
thr_m = 0.0; % default: even s=0 does not exceed failure criterion
for s = scale_steps
% Weaken only muscle m across all time samples
Force_tmp = Force_sel_i;
Force_tmp(m,:) = s * Force_sel_i(m,:);
% Recompute subject-level failure ratio under this single-muscle weakening
fail_ratio = evaluate_fail_ratio_simple(MA3_sel_i, Force_tmp, NeededTorque, timeIdxToUse);
% First s that pushes fail_ratio above 0.20 is the frailty threshold
if fail_ratio > fail_threshold
thr_m = s;
break;
end
end
threshold(m) = thr_m;
end
end
```

Code 4 Main Monte Carlo frailty pipeline (R/F variability)

```
%% ============================================================
%% Main Monte Carlo frailty pipeline (R/F variability)
%% ============================================================
% Purpose:
% Estimate population-level frailty (failure probability) under modeled inter-subject
% variability in (i) muscle moment arms and (ii) muscle force capacity, then optionally
% compute muscle-specific thresholds and sensitivity metrics.
%
% Textual definitions (embedded):
% - Inter-subject variability model (muscle-wise multiplicative scaling):
```

```
% r_i(t) <- kR_i * r_i(t) (moment arm scaling per muscle)
% F_i(t) <- kF_i * F_i(t) (force capacity scaling per muscle)
% where kR_i ~ Normal(1, sigmaR), kF_i ~ Normal(1, sigmaF), then truncated to
% physiologically plausible bounds to avoid negative/extreme values.
%
% - For each virtual subject, compute:
% fail_ratio, isFail using Appendix B.
%
% - Population frailty index (failure probability):
% FI = (number of subjects with isFail==true) / N
%
% - Muscle selection:
% Only include muscles with non-negligible moment arms (>tol_ma) for at least one
% of the 3 target DOFs at any time point (removes irrelevant muscles).
% --- User-defined parameters ---
N = 1000; % number of virtual subjects
sigmaR = 0.10; % SD for moment-arm scaling (10%)
sigmaF = 0.50; % SD for force scaling (50%)
tol_ma = 1e-6; % moment-arm non-zero threshold
% --- Inputs expected to be already loaded/constructed ---
% MomentArm : [7 x 92 x nTime] moment arms for all DOFs (m)
% Force0 : [92 x nTime] baseline muscle force capacity (N)
% Time : [nTime x 1] time vector
% idx_dof : [1 x 3] indices of the 3 target DOFs in MomentArm
% data_id : inverse dynamics table data
% time_id : inverse dynamics time vector
% idx_id_cols : [1 x 3] column indices for target torques in inverse dynamics
% timeIdxToUse : indices of frames used in evaluation (subset of 1:nTime)
% --- Step 0: Extract target DOFs and select relevant muscles ---
MA3 = MomentArm(idx_dof, :, :); % [3 x 92 x nTime]
keepMuscleMask = any(abs(MA3) > tol_ma, [1 3]); % [1 x 92], muscle contributes to ≥1 DOF at ≥1
time
idxMuscles = find(keepMuscleMask);
MA3_sel0 = MA3(:, idxMuscles, :); % [3 x nSel x nTime]
Force_sel0 = Force0(idxMuscles, :); % [nSel x nTime]
nMusSel = numel(idxMuscles);
% --- Step 0 (cont.): Build NeededTorque from inverse dynamics and align to Time grid ---
TorqueID_raw = data_id(:, idx_id_cols); % [nTime_id x 3], units N·m
TorqueID_interp = interp1(time_id, TorqueID_raw, Time, 'linear', 'extrap');
NeededTorque = TorqueID_interp.'; % [3 x nTime], units N·m
% --- Step A: Monte Carlo simulation over R and F ---
fail_ratio_all = zeros(N,1); % per-subject failure ratio
isFail_all = false(N,1); % per-subject binary failure label
for i = 1:N
% 1) Sample muscle-wise scaling factors for this virtual subject
kR = 1 + sigmaR * randn(nMusSel,1); % moment-arm scaling
kF = 1 + sigmaF * randn(nMusSel,1); % force-capacity scaling
% 2) Truncate to plausible bounds (avoid negative/extreme values)
kR = min(max(kR, 0.5), 1.5); % R scaling in [0.5, 1.5]
kF = min(max(kF, 0.1), 3.0); % F scaling in [0.1, 3.0]
% 3) Apply scaling to generate subject-specific R(t) and F(t)
MA3_sel_i = MA3_sel0 .* reshape(kR', [1, nMusSel, 1]); % [3 x nSel x nTime]
Force_sel_i = Force_sel0 .* kF; % [nSel x nTime]
% 4) Evaluate infeasibility over time and summarize as failure ratio / failure label
[fail_ratio, isFail] = evaluate_fail_ratio_simple( ...
MA3_sel_i, Force_sel_i, NeededTorque, timeIdxToUse);
fail_ratio_all(i) = fail_ratio;
```

```
isFail_all(i) = isFail;
% Optional (muscle thresholds for this subject; Appendix C)
% threshold_i = compute_muscle_thresholds(MA3_sel_i, Force_sel_i, NeededTorque, timeIdxToUse);
end
% Population frailty index (failure probability)
FI = sum(isFail_all) / N;
```

Code 5 Sensitivity analysis (muscle importance)

```
%% ============================================================
%% Sensitivity analysis (muscle importance)
%% ============================================================
% Purpose:
% Identify muscles whose modeled variability is most strongly associated with task
% infeasibility, using association-based sensitivity metrics.
%
% Textual definitions (embedded):
% - Outputs from Monte Carlo:
% kF_all : [N x nMusSel] force scaling factors
% kR_all : [N x nMusSel] moment-arm scaling factors
% fail_ratio_all : [N x 1] continuous outcome (0~1)
% isFail_all : [N x 1] binary failure label (fail_ratio > 0.20)
%
% - (E1) Correlation screening:
% corr_F(m) = corr(kF_m, fail_ratio) (rank muscles by |corr_F|)
%
% - (E2) Linear regression on continuous fail_ratio:
% Standardize predictors (z-score each column), fit:
% fail_ratio ~ 1 + z([kF, kR])
% Coefficients are comparable due to standardization. A combined importance index:
% importance(m) = sqrt(beta_F(m)^2 + beta_R(m)^2)
% Report R^2 as explained variance.
%
% - (E3) Logistic regression on binary failure (optional):
% isFail ~ 1 + z(kF_topK)
% Coefficients are log-odds; p-values (if reported) are descriptive and model-dependent.
%
% Interpretation note:
% These are association-based sensitivities under the assumed variability model, not causal
claims.
%% --- E1: Correlation screening (kF vs fail_ratio) ---
corr_F = zeros(nMusSel, 1);
for m = 1:nMusSel
corr_F(m) = corr(kF_all(:,m), fail_ratio_all, 'rows','complete');
end
[~, idxSortCorr] = sort(abs(corr_F), 'descend');
%% --- E2: Linear regression (fail_ratio ~ 1 + z([kF, kR])) ---
X = [kF_all, kR_all]; % [N x (2*nMusSel)]
y = fail_ratio_all; % [N x 1]
% z-score each predictor column
Xz = X;
for j = 1:size(X,2)
mu = mean(X(:,j)); sd = std(X(:,j));
if sd > 0, Xz(:,j) = (X(:,j) - mu) / sd; else, Xz(:,j) = 0; end
```

```
end
% OLS regression and goodness-of-fit
[B_lin,~,~,~,stats_lin] = regress(y, [ones(size(Xz,1),1), Xz]);
R2 = stats_lin(1);
beta_all = B_lin(2:end);
beta_F = beta_all(1:nMusSel);
beta_R = beta_all(nMusSel+1:end);
importance = sqrt(beta_F.^2 + beta_R.^2);
[~, idxSortImp] = sort(importance, 'descend');
%% --- E3: Logistic regression on binary failure (optional; top-K by correlation) ---
try
topK = min(10, nMusSel);
idxTop = idxSortCorr(1:topK);
X_log = kF_all(:, idxTop);
y_log = isFail_all;
% z-score predictors
Xz_log = X_log;
for j = 1:size(X_log,2)
mu = mean(X_log(:,j)); sd = std(X_log(:,j));
if sd > 0, Xz_log(:,j) = (X_log(:,j) - mu) / sd; else, Xz_log(:,j) = 0; end
end
% Logistic regression: isFail ~ 1 + z(kF_topK)
[B_log, ~, stats_log] = glmfit(Xz_log, y_log, 'binomial', 'link','logit');
beta_log = B_log(2:end); % log-odds coefficients
p_log = stats_log.p(2:end); % descriptive p-values (optional)
catch
% If toolbox is unavailable, skip logistic regression.
end
```

References

[1] Cohn BA, et al. (2018), Front Comput Neurosci;

[2] Reznick E, et al. (2021), Sci Data;

[3] Kutch JJ, Valero-Cuevas FJ. (2011), J Biomech;

[4] Chen Z, Franklin D. (2025), J Biomech;

[5] Ward SR, et al. (2009), Clin Orthop Relat Res;

[6] Fried LP, et al. (2001), J Gerontol A Biol Sci Med Sci;

[7] Panhwar YN, et al. (2019), BMC Biomed Eng;

[8] Protopapadaki A, et al. (2007), Clin Biomech;

[9] Verghese J, et al. (2008), Arch Phys Med Rehabil;

[10] Kwei, E. Feasible Torque/Force Sets Neuromechanics GUI [Computer software]. https://github.com/ek-usc/FrailtyProject-StairClimbingModelGUI/tree/master;